QUIZ PERTEMUAN 5

PEMROGRAMAN BERORIENTASI OBJEK

Oleh:

Zeyan Fahluzi Alhariz

250660221033

SOAL

1. Konsep Polymorphism

Jelaskan pengertian polymorphism dalam OOP dan berikan satu contoh method yang menghasilkan perilaku berbeda pada class yang berbeda.

Jawaban:

- Pengertian Polymorphism: Dalam OOP, *polymorphism* (polimorfisme/banyak bentuk) adalah kemampuan suatu objek dari kelas yang berbeda-beda untuk merespons pemanggilan nama *method* yang sama dengan cara atau perilaku spesifik mereka masing-masing. Program tidak perlu mengetahui tipe objek secara spesifik, cukup mengetahui bahwa objek tersebut memiliki *method* yang dimaksud.
- Contoh Penerapan: *Method* `hitung_luas()`. Jika *method* ini dipanggil pada objek dari *class* `Persegi`, ia akan mengalikan sisi dengan sisi. Namun, jika dipanggil pada objek dari *class* `Lingkaran`, ia akan mengalikan pi dengan kuadrat jari-jari. Nama *method*-nya sama, tetapi perilakunya berbeda sesuai dengan *class*-nya.

2. Overriding dan Polymorphism

Jelasakan hubungan antara overriding dan polymorphism. Mengapa overriding menjadi dasar perilaku polimorfik pada python?

Jawaban:

- Hubungan: *Overriding* (menimpa *method* induk) adalah salah satu mekanisme utama untuk mewujudkan *polymorphism* dalam pewarisan (*inheritance*). Dengan *overriding*, sebuah *subclass* dapat menyediakan wujud implementasi spesifik dari *method* umum yang didefinisikan di *superclass*.
- Alasan menjadi dasar di Python: Python bersifat dinamis (*dynamic typing* / *duck typing*). Saat sebuah *method* dipanggil, Python akan mencari eksekusi *method* pada level objek (*subclass*) terlebih dahulu saat *runtime*. Jika *method* tersebut telah di-*override*, Python akan menjalankan versi *subclass* tersebut tanpa memedulikan tipe *superclass*-nya, sehingga menciptakan perilaku polimorfik secara otomatis.

3. Prediksi Output

```python
class Pengguna:
    def tampilkan_peran(self):
        return "Pengguna sistem"

class Mahasiswa(Pengguna):
    def tampilkan_peran(self):
        return "Mahasiswa"

class Dosen(Pengguna):
    def tampilkan_peran(self):
        return "Dosen"

for data in [Mahasiswa(), Dosen()]:
    print(data.tampilkan_peran())
```

Tuliskan output dari program tersebut dan jelaskan konsep OOP yang ditunjukan.

Jawaban:

Output:

Mahasiswa

Dosen

Penjelasan Konsep OOP yang Ditunjukkan:

- Di dalam iterasi `for data in [Mahasiswa(), Dosen()]:`, program memanggil *method* `tampilkan_peran()` pada objek yang berbeda tanpa mengecek tipe *class*-nya secara manual.
- Karena *class* `Mahasiswa` dan `Dosen` sama-sama melakukan *overriding* terhadap *method* milik *superclass* `Pengguna`, pemanggilan tersebut secara cerdas merespons dengan nilai kembalian (*return*) spesifik dari masing-masing *class*, yaitu `"Mahasiswa"` dan `"Dosen"`.

4. Duck Typing

```
class Mahasiswa:
    def ringkasan(self):
        return "Mahasiswa – Budi"

class Dosen:
    def ringkasan(self):
        return "Dosen – Sari"

def tampilkan(data):
    print(data.ringkasan())

tampilkan(Mahasiswa())
tampilkan(Dosen())
```

Jelaskan mengapa fungsi tampilkan() dapat menerima kedua object meskipun mahasiswa dan dosen tidak memiliki hubungan inheritance.

Jawaban:

Fungsi `tampilkan(data)` dapat menerima objek `Mahasiswa` maupun `Dosen` meskipun keduanya tidak memiliki hubungan *inheritance* (pewarisan) karena Python menerapkan konsep yang disebut Duck Typing.

Dalam *Duck Typing*, tipe atau *class* dari sebuah objek tidaklah penting; yang diperdulikan oleh Python hanyalah apakah objek tersebut memiliki *method* atau atribut yang sedang dipanggil. Karena kelas `Mahasiswa` dan `Dosen` sama-sama memiliki *method* bernama `ringkasan()`, maka fungsi `tampilkan()` dapat mengeksekusi `data.ringkasan()` pada kedua objek tersebut tanpa mempermasalahkan dari *class* mana mereka berasal. *(Konsep ini berasal dari pepatah: "Jika ia berjalan seperti bebek dan bersuara seperti bebek, maka ia adalah bebek").*

5. *Merancang Fungsi polimorfik*

Tuliskan fungsi tampilkan_semua(daftar) yang menerima daftar object beragam dan memanggil method ringkasan() oada setiap object tanpa memeriksa tipe. Jelaskan apa yang terjadi jika sebuah object tidak memiliki method ringkasan().

Jawaban:

Fungsi tampilkan _semua(daftar):

```
def tampilkan_semua(daftar):
    for objek in daftar:
        print(objek.ringkasan())
```

Apa yang terjadi jika sebuah object tidak memiliki method `ringkasan()`? Jika fungsi ini menerima daftar (list) yang di dalamnya terdapat objek yang tidak memiliki *method* `ringkasan()`, program akan berhenti dan menghasilkan *error* pada saat *runtime*, secara spesifik `AttributeError`. Pesan *error* yang muncul biasanya berbunyi seperti: `'NamaClass' object has no attribute 'ringkasan'`.